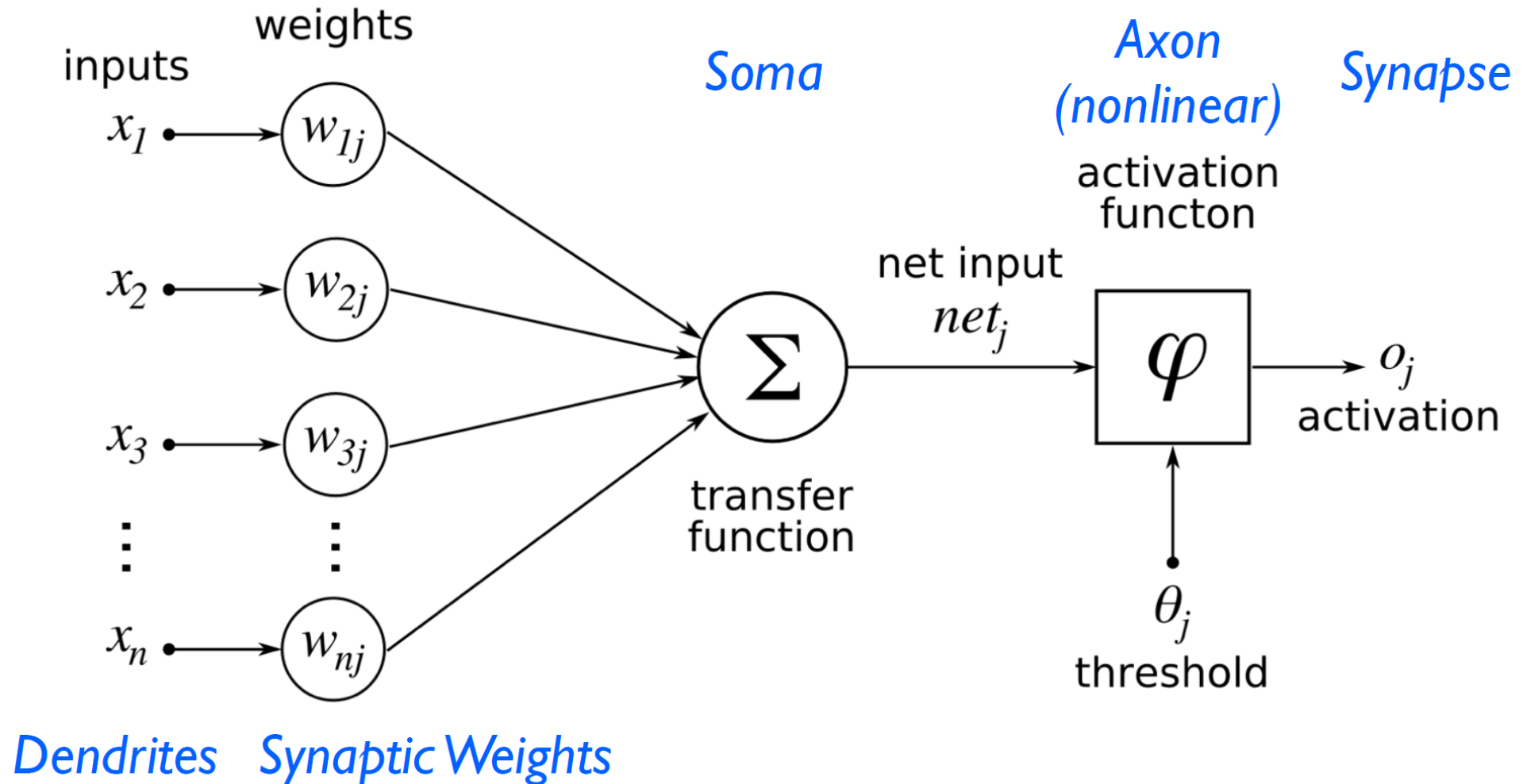


➤ NEURAL NETWORK STRUCTURES

- McCulloch-Pitts Model
- Unipolar/bipolar case
- Linear Separation
- Logic Functions
- Basic Limitation : EXOR Function
- General Case
- Multilayer Structures : Feedforward Case

McCulloch-Pitts Neuron



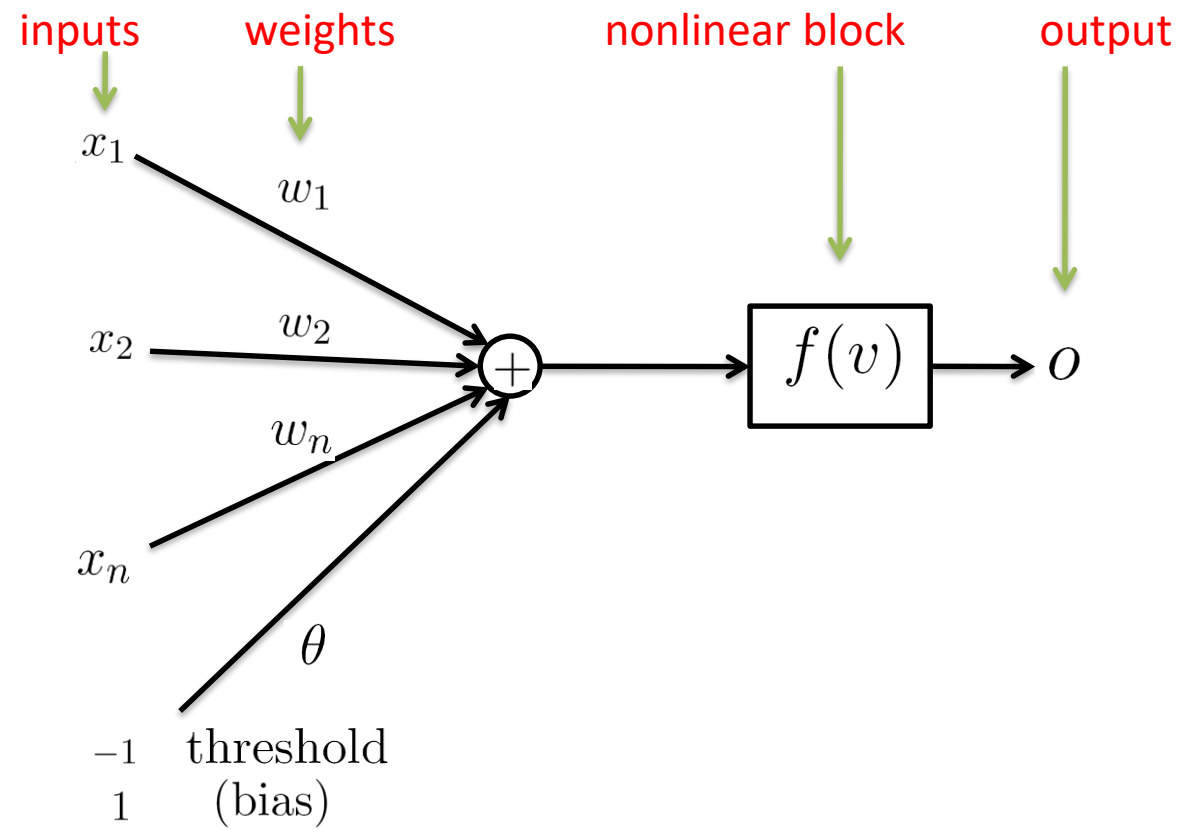
A LOGICAL CALCULUS OF THE
IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH AND WALTER PITTS

FROM THE UNIVERSITY OF ILLINOIS, COLLEGE OF MEDICINE,
DEPARTMENT OF PSYCHIATRY AT THE ILLINOIS NEUROPSYCHIATRIC INSTITUTE,
AND THE UNIVERSITY OF CHICAGO

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.

I. Introduction

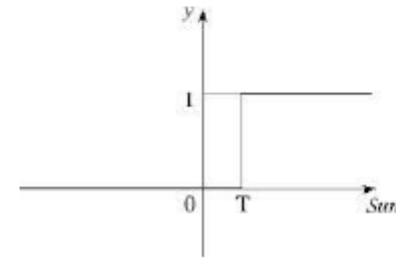


- $\Sigma = w_1x_1 + w_2x_2 + \dots + w_nx_n, v = \Sigma \pm \theta \rightarrow o = f(v).$

Basic assumptions : (unipolar case)

- $w_i = \pm 1$ (+1 for excitatory synapse, -1 for inhibitory synapse)
- $x_i = 1$ or 0 (digital analogy : +1 : signal, 0 : no signal)
- $f(v)$ has special form : **threshold function, hard limiter**

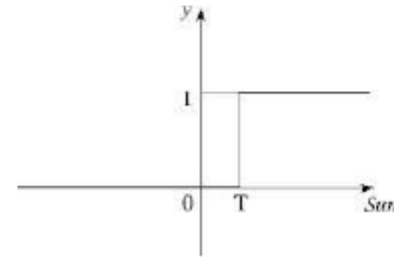
$$f(v) = \begin{cases} +1 & v \geq 0 \\ 0 & v < 0 \end{cases}$$



Basic assumptions : (unipolar case)

- $w_i = \pm 1$ (+1 for excitory synapse, -1 for inhibitory synapse)
- $x_i = 1$ or 0 (digital analogy : +1 : signal, 0 : no signal)
- $f(v)$ has special form : **reshold function, hard limiter**

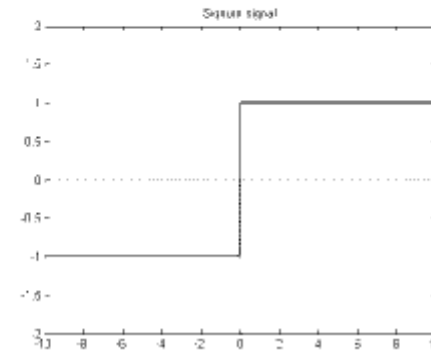
$$f(v) = \begin{cases} +1 & v \geq 0 \\ 0 & v < 0 \end{cases}$$



• Simple extension : (bipolar case)

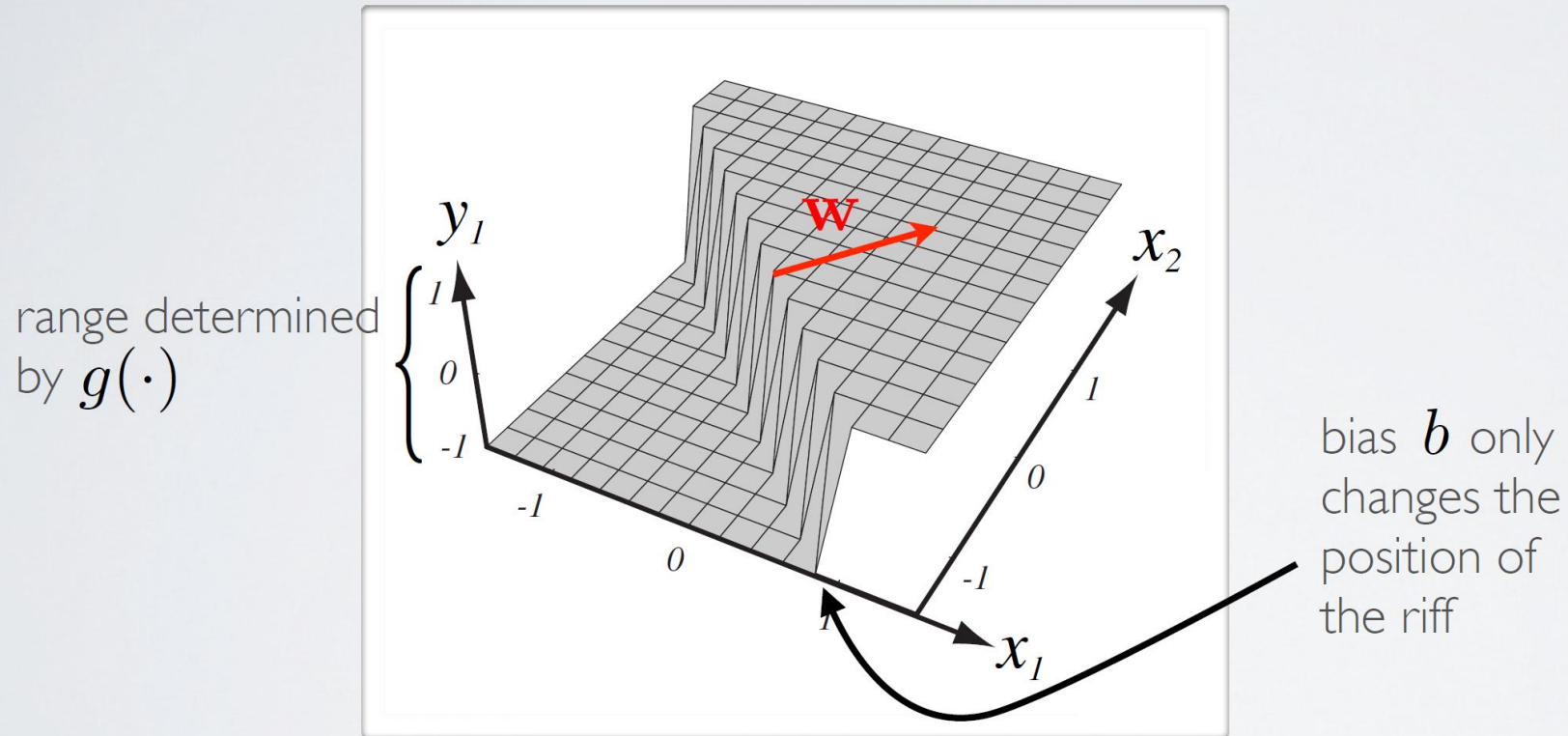
- $x_i = \pm 1$ (digital analogy : +1 : signal, -1 : no signal)
- $f(v)$ has a special form : **ignum function**

$$f(v) = \begin{cases} +1 & v \geq 0 \\ -1 & v < 0 \end{cases}$$



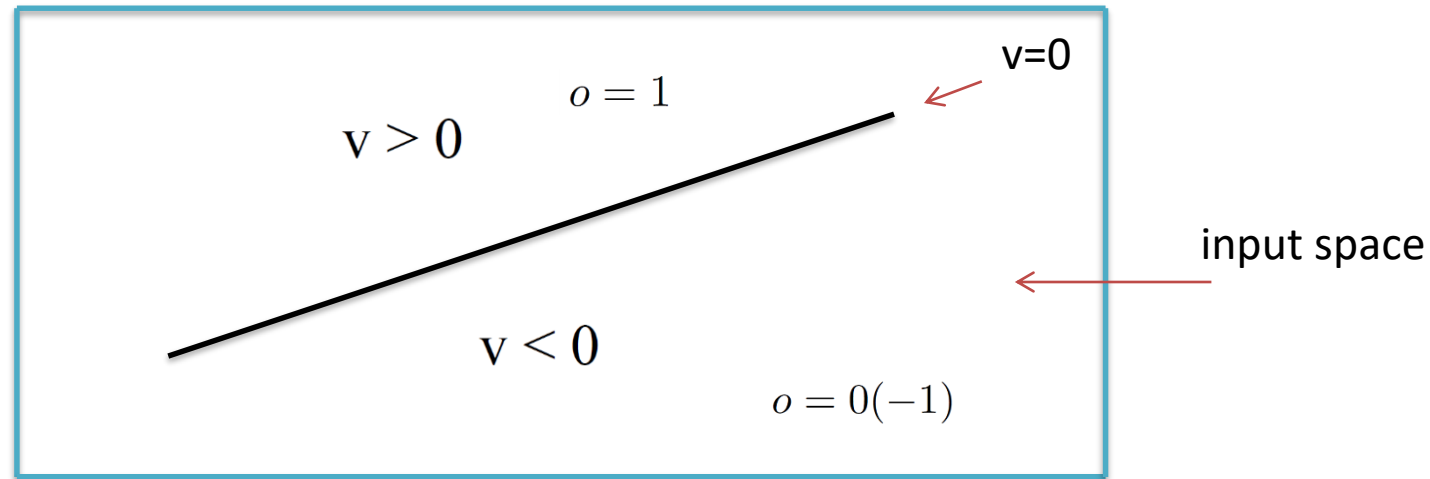
ARTIFICIAL NEURON

Topics: connection weights, bias, activation function



(from Pascal Vincent's slides)

- **Plane analogy:** $v = 0$ line (hyperplane for higher dimensions) separates the input space into two parts.



• **Conclusion:** If the patterns remain in one side for one class of inputs, and on the other side for the second class $\rightarrow$ separation is possible by looking at the output $\rightarrow$ **Linear Separability**

Implementing Logic Gates with MP Neurons

We can use McCulloch-Pitts neurons to implement the basic logic gates (e.g. AND, OR, NOT).

It is well known from logic that we can construct any logical function from these three basic logic gates.

All we need to do is find the appropriate connection weights and neuron thresholds to produce the right outputs for each set of inputs.

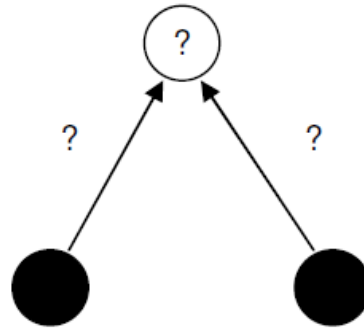
We shall see explicitly how one can construct simple networks that perform NOT, AND, and OR.

Implementation of Logical NOT, AND, and OR

NOT	
in	out
0	1
1	0

AND		
in ₁	in ₂	out
0	0	0
0	1	0
1	0	0
1	1	1

OR		
in ₁	in ₂	out
0	0	0
0	1	1
1	0	1
1	1	1



Problem: Train network to calculate the appropriate weights and thresholds in order to classify correctly the different classes (i.e. form **decision boundaries** between classes).

Implementation of Logical NOT, AND, and OR

In each case, we have inputs in_i and outputs out , and need to determine appropriate weights and thresholds. It is easy to find solutions by inspection:

NOT

in	out
0	1
1	0

AND

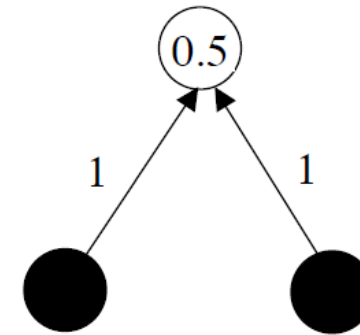
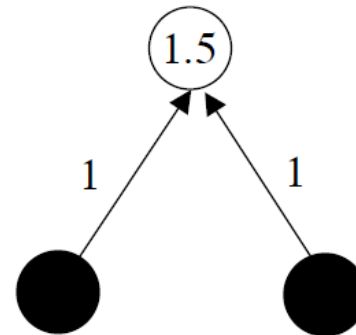
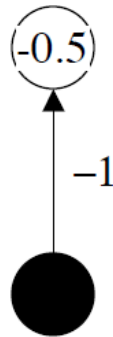
in_1	in_2	out
0	0	0
0	1	0
1	0	0
1	1	1

OR

in_1	in_2	out
0	0	0
0	1	1
1	0	1
1	1	1

Thresholds $\Rightarrow$

Weights $\Rightarrow$



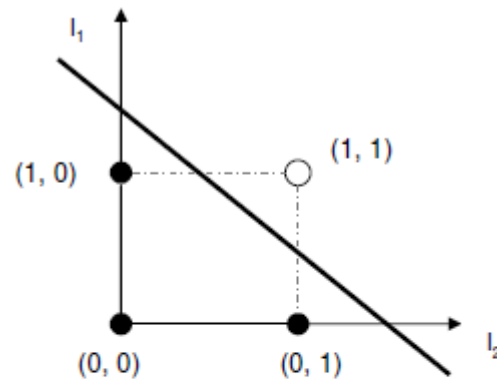
Decision Boundaries for AND and OR

We can now plot the decision boundaries of our logic gates

AND

$w_1=1, w_2=1, \theta=1.5$

AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



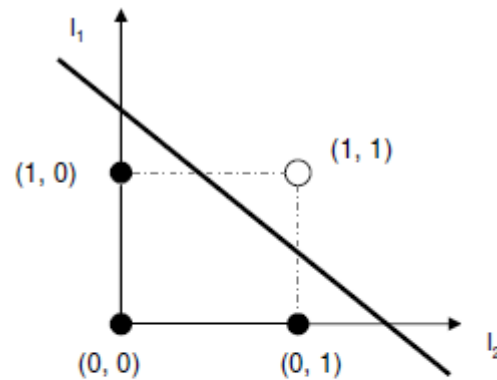
Decision Boundaries for AND and OR

We can now plot the decision boundaries of our logic gates

AND

$w_1=1, w_2=1, \theta=1.5$

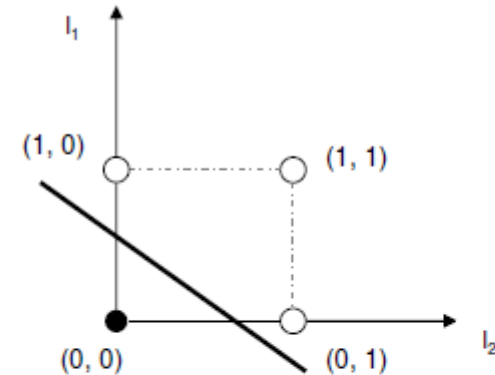
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



OR

$w_1=1, w_2=1, \theta=0.5$

OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1



Finding Weights Analytically for the AND Network

We have two weights w_1 and w_2 and the threshold θ , and for each training pattern we need to satisfy:

$$out = step(w_1 in_1 + w_2 in_2 - \theta)$$

So the training data lead to four inequalities:

in_1	in_2	out
0	0	0
0	1	0
1	0	0
1	1	1

 $\Rightarrow$

Finding Weights Analytically for the AND Network

We have two weights w_1 and w_2 and the threshold θ , and for each training pattern we need to satisfy:

$$out = step(w_1 in_1 + w_2 in_2 - \theta)$$

So the training data lead to four inequalities:

in_1	in_2	out	$\Rightarrow$	
0	0	0		$w_1 0 + w_2 0 - \theta < 0$
0	1	0		$w_1 0 + w_2 1 - \theta < 0$
1	0	0		$w_1 1 + w_2 0 - \theta < 0$
1	1	1		$w_1 1 + w_2 1 - \theta \geq 0$

Finding Weights Analytically for the AND Network

We have two weights w_1 and w_2 and the threshold θ , and for each training pattern we need to satisfy:

$$out = step(w_1 in_1 + w_2 in_2 - \theta)$$

So the training data lead to four inequalities:

in_1	in_2	out
0	0	0
0	1	0
1	0	0
1	1	1

 $\Rightarrow$

$w_1 0 + w_2 0 - \theta < 0$
$w_1 0 + w_2 1 - \theta < 0$
$w_1 1 + w_2 0 - \theta < 0$
$w_1 1 + w_2 1 - \theta \geq 0$

 $\Rightarrow$

$\theta > 0$
$w_2 < \theta$
$w_1 < \theta$
$w_1 + w_2 \geq \theta$

It is easy to see that there are an infinite number of solutions. Similarly, there are an infinite number of solutions for the NOT and OR networks.

- Basic logic operations are **LINEARLY SEPARABLE**
-  **could be realized by McCulloch-Pitts neuron!**

UNIPOLAR CASE ($x_i = 1$ or 0 , threshold fn.)

- **Example 1** : Logic AND operation : ($i = 1, 2, \dots, n$)

$$o = x_1 \wedge x_2 \wedge \dots \wedge x_n$$

$$o = f(w_1x_1 + w_2x_2 + \dots + w_nx_n - \theta)$$

$$o = \begin{cases} 1 & \text{all } x_i = 1 \\ 0 & \text{at least one } x_j = 0 \end{cases}$$

$$\text{Choose } w_i = 1 \implies \begin{cases} n - \theta \geq 0 \\ n - 1 - \theta < 0 \end{cases} \implies n - 1 < \theta \leq n$$

- **Example 2** : Logic OR operation : $(i = 1, 2, \dots, n)$

$$o = x_1 \vee x_2 \vee \dots \vee x_n$$

$$o = f(w_1x_1 + w_2x_2 + \dots + w_nx_n - \theta)$$

$$o = \begin{cases} 1 & \text{at least } x_j = 1 \\ 0 & \text{all } x_i = 0 \end{cases}$$

$$\text{Choose } w_i = 1 \implies \begin{cases} 1 - \theta \geq 0 \\ -\theta < 0 \end{cases} \implies 0 < \theta \leq 1$$

- **Example 3** : Logic NOT operation : ($i = 1$)

$$o = \bar{x}$$

$$o = \begin{cases} 1 & x = 0 \\ 0 & x = 1 \end{cases} \quad o = f(wx - \theta)$$

$$\text{Choose } w = -1 \implies \begin{cases} -1 - \theta < 0 \\ -\theta \geq 0 \end{cases} \implies -1 < \theta \leq 0$$

- All (unipolar) logic operations can be realized by (possibly cascading) McCulloch-Pitts neuron model !
- Relation with Boolean Algebra $\Rightarrow$ Computers $\Rightarrow$ Inspired von Neumann

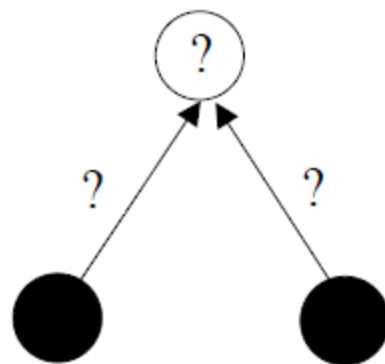
- All of these operations are **LINEARLY SEPARABLE** !

The Need to Find Weights Analytically

Constructing simple networks by hand (e.g., by trial and error) is one thing. But what about harder problems? For example, what about:

XOR

in_1	in_2	out
0	0	0
0	1	1
1	0	1
1	1	0



How long should we keep looking for a solution? We need to be able to calculate appropriate parameter values rather than searching for solutions by trial and error.

Each training pattern produces a linear inequality for the output in terms of the inputs and the network parameters. These can be used to compute the weights and thresholds.

- What can/cannot we perform with a SINGLE NEURON?
- Basic limitation is LINEAR SEPARABILITY

EXOR operation (EXclusive OR) ($n = 2$, $x_i = 1$ or 0)

$$x_1 \oplus x_2 = \begin{cases} +1 & \text{if } x_1 \neq x_2 \\ 0 & \text{if } x_1 = x_2 \end{cases}$$

- Can we find a SINGLE NEURON realizing this logic function ?
- (Note that with more than a single neuron, this is always possible!)

- $o = f(w_1x_1 + w_2x_2 - \theta)$

Limitations of Simple Perceptrons

We can follow the same procedure for the XOR network:

in_1	in_2	out
0	0	0
0	1	1
1	0	1
1	1	0

 $\Rightarrow$

$w_1 0 + w_2 0 - \theta < 0$
$w_1 0 + w_2 1 - \theta \geq 0$
$w_1 1 + w_2 0 - \theta \geq 0$
$w_1 1 + w_2 1 - \theta < 0$

Limitations of Simple Perceptrons

We can follow the same procedure for the XOR network:

in_1	in_2	out
0	0	0
0	1	1
1	0	1
1	1	0

 $\Rightarrow$

$w_1 0 + w_2 0 - \theta < 0$
$w_1 0 + w_2 1 - \theta \geq 0$
$w_1 1 + w_2 0 - \theta \geq 0$
$w_1 1 + w_2 1 - \theta < 0$

 $\Rightarrow$

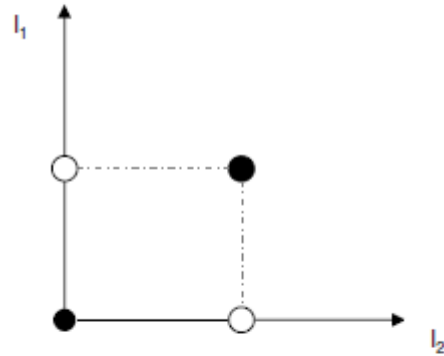
$\theta > 0$
$w_2 \geq \theta$
$w_1 \geq \theta$
$w_1 + w_2 < \theta$

Clearly the first, second and third inequalities are incompatible with the fourth, so there is in fact no solution. We need more complex networks, e.g. that combine together many simple networks, or use different activation/thresholding/transfer functions.

Decision Boundary for XOR

The difficulty in dealing with XOR is rather obvious. We need two straight lines to separate the different outputs/decisions:

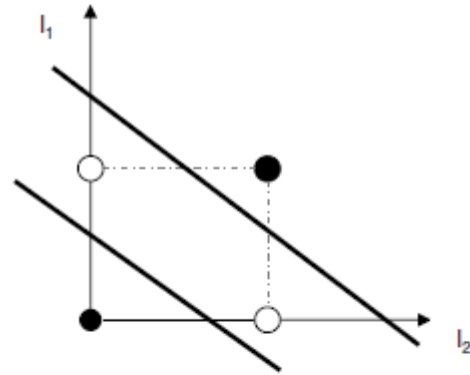
XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



Decision Boundary for XOR

The difficulty in dealing with XOR is rather obvious. We need two straight lines to separate the different outputs/decisions:

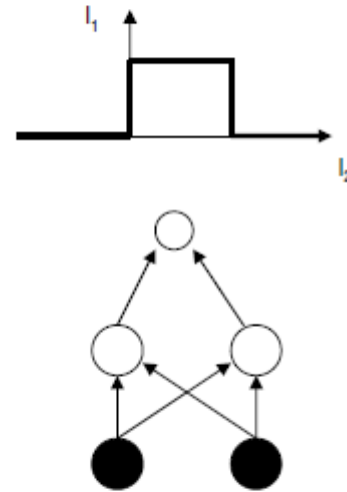
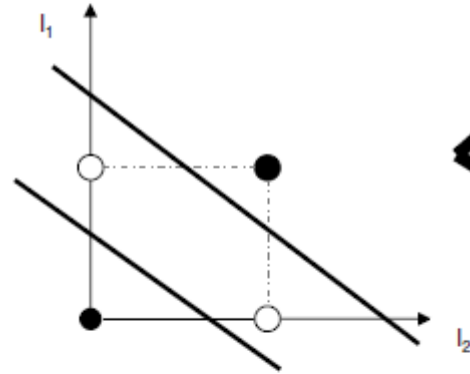
XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



Decision Boundary for XOR

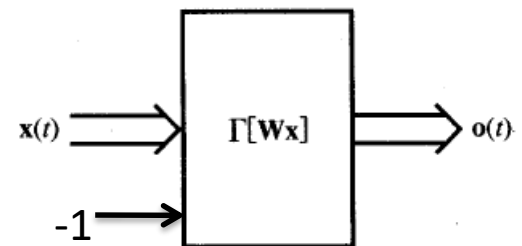
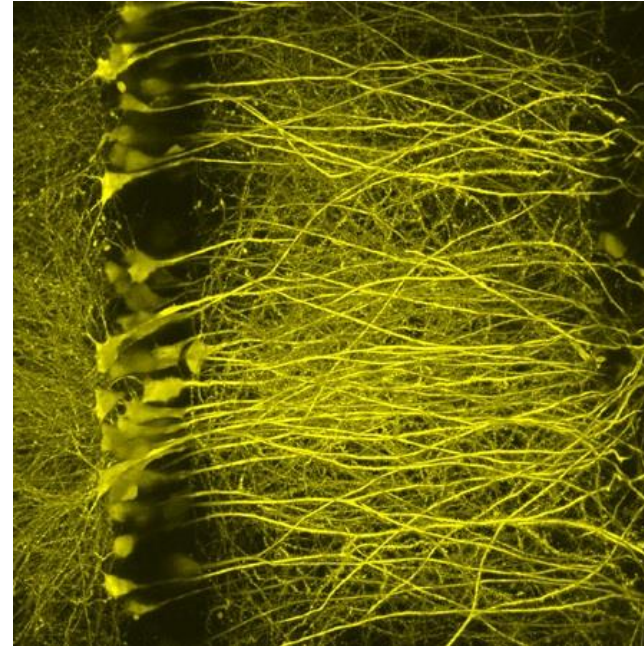
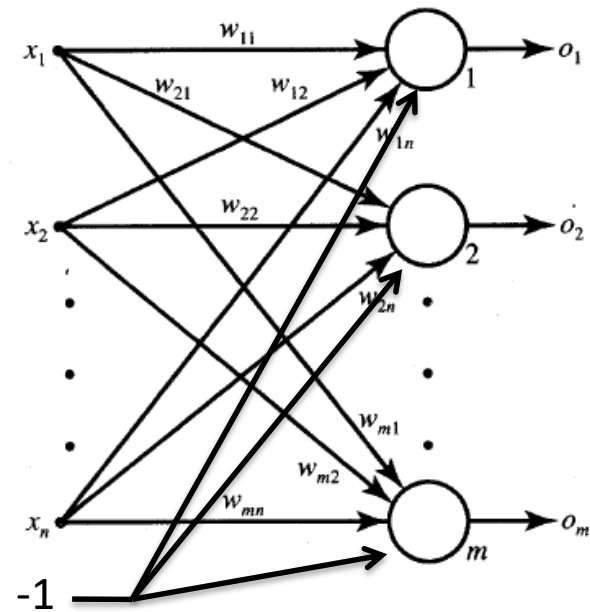
The difficulty in dealing with XOR is rather obvious. We need two straight lines to separate the different outputs/decisions:

XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0

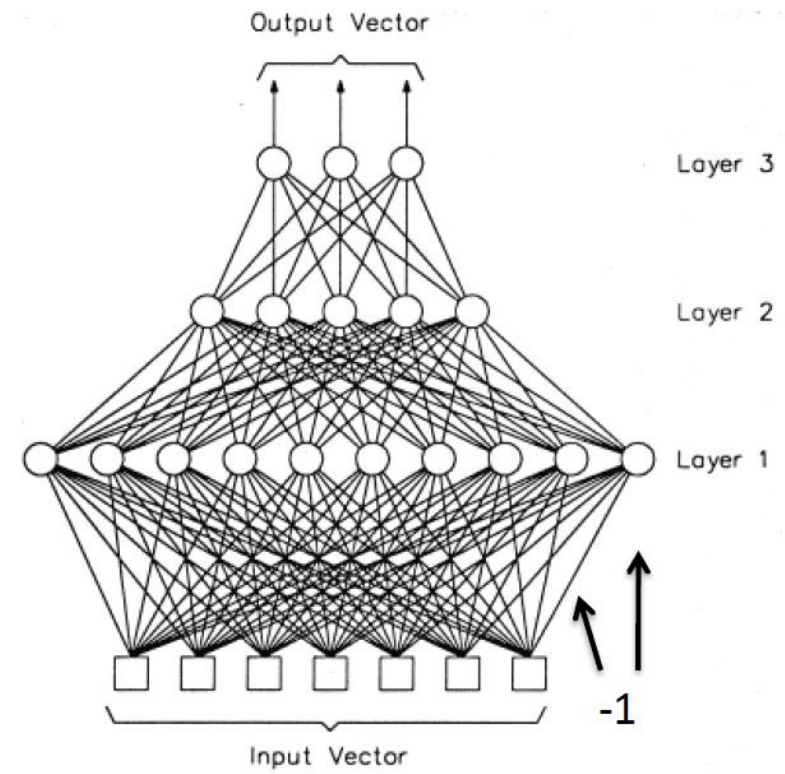
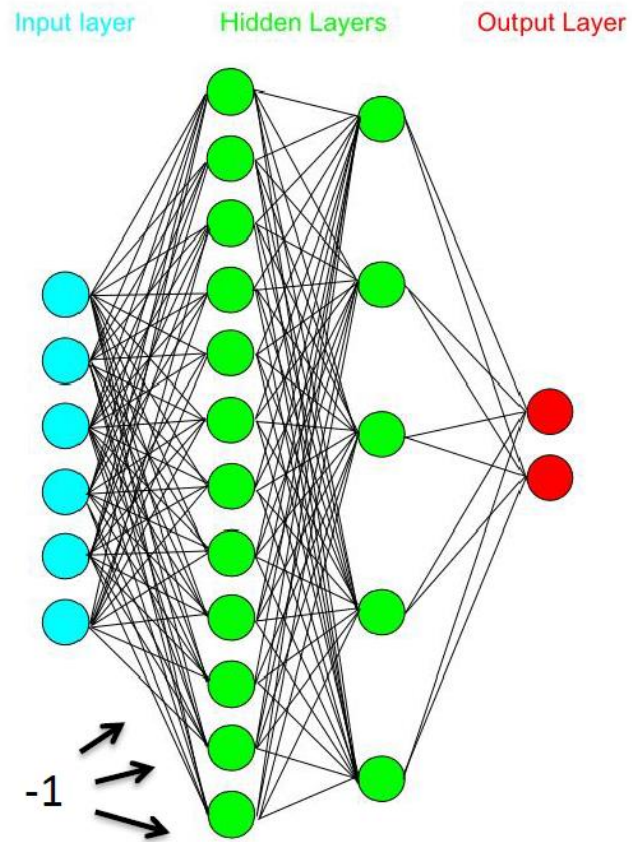


Solution: either change the transfer function so that it has more than one decision boundary, or use a more complex network that is able to generate more complex decision boundaries.

- Single neuron is not sufficient !
- One Extension : **SINGLE LAYER NETWORK**

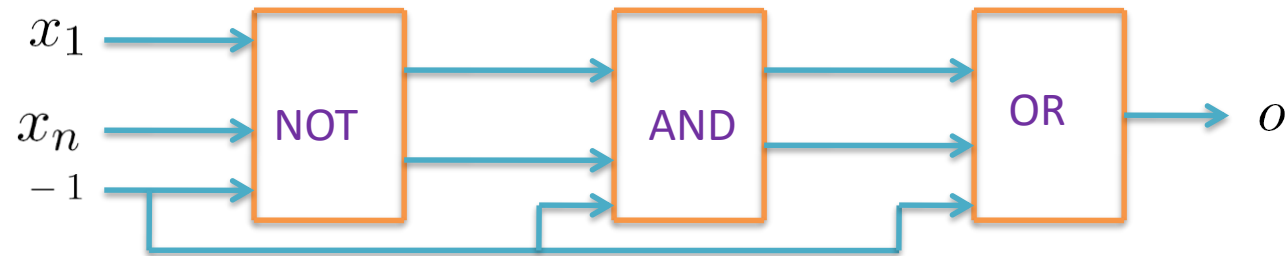


• MULTILAYER NEURAL NETWORKS



- **Claim** : Any (unipolar) logic operation can be realized by at most 3 layers of neurons !

$$O = (x_1 \wedge \bar{x}_2 \dots) \vee (\bar{x}_1 \wedge \dots) \vee \dots$$



- order of blocks may change!
 - not unique!
 - how many neurons do we have in each block?
-
- Can we eliminate NOT block? (weight is -1 !)

- Why multilayer?
- Arbitrary logic function realization....
- Single Neuron : LINEAR SEPARATION
- Remember EXOR Function...

We can realize each output separately...

x_1	x_2	o_1	o_2
0	0	0	0
0	1	1	0
1	0	0	1
1	1	0	0

$\downarrow$

$\leftarrow$

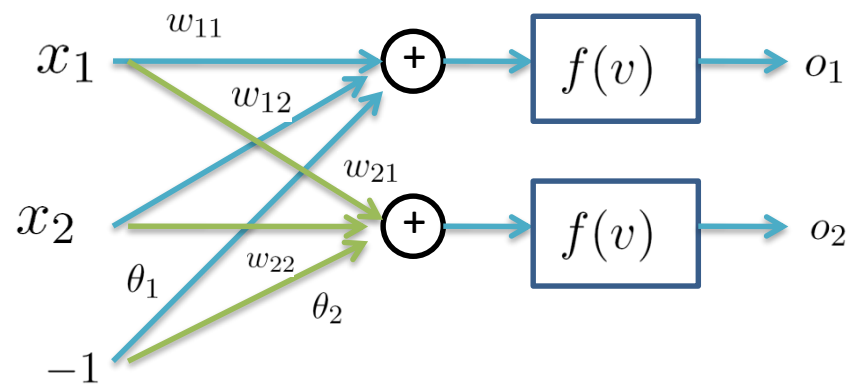
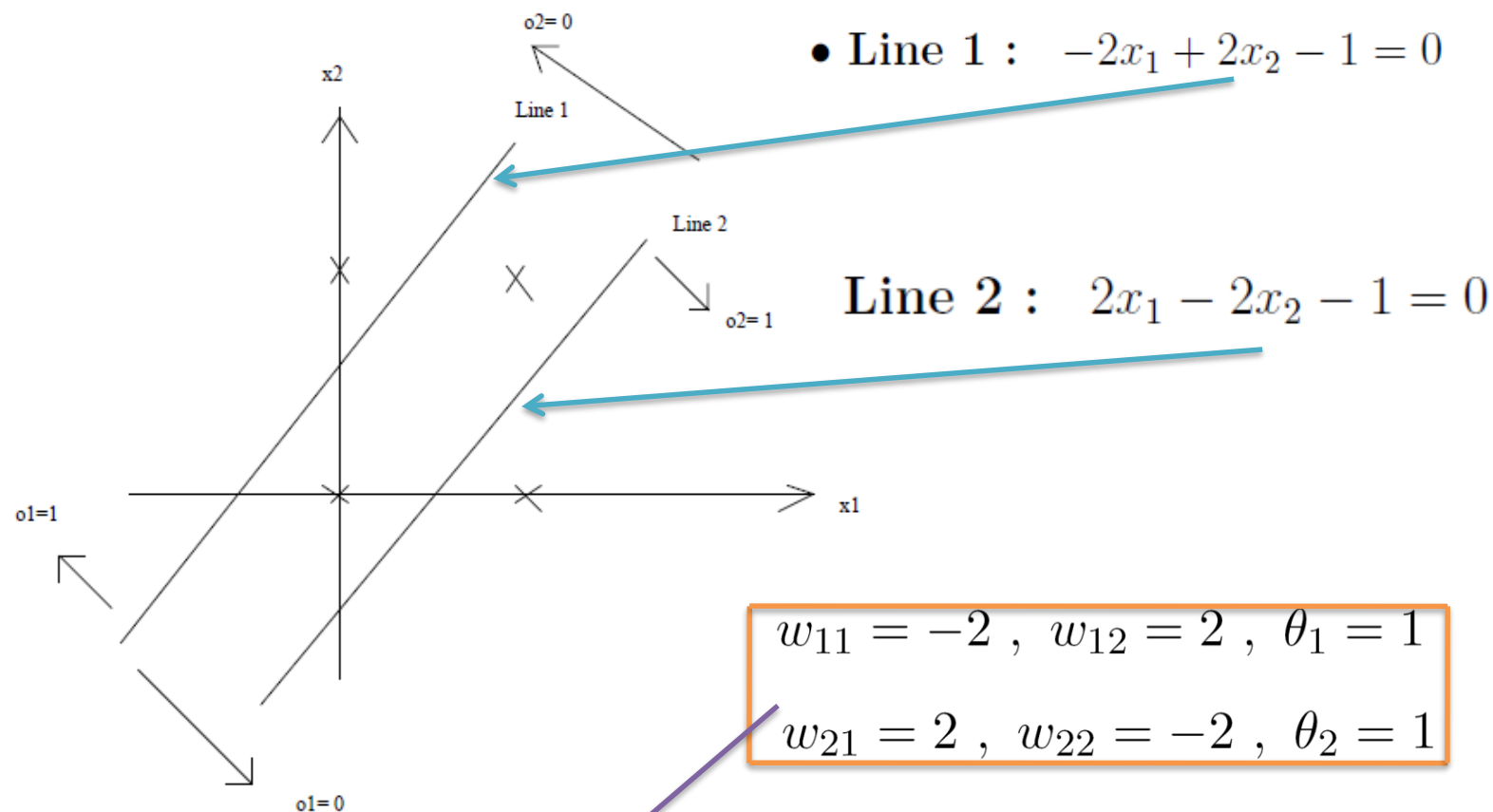
$o_1 = \bar{x}_1 \wedge x_2$
 $o_2 = x_1 \wedge \bar{x}_2$

}

logically

- $v_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 = w_{11}x_1 + w_{12}x_2 - \theta_1 \rightarrow o_1 = f(v_1)$
- $v_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 = w_{21}x_1 + w_{22}x_2 - \theta_2 \rightarrow o_2 = f(v_2)$

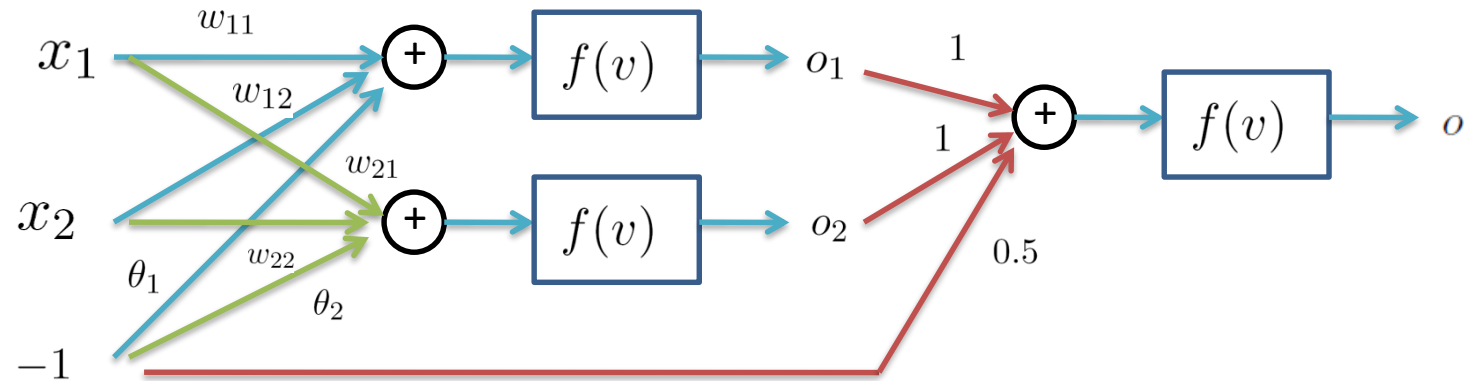
analytically



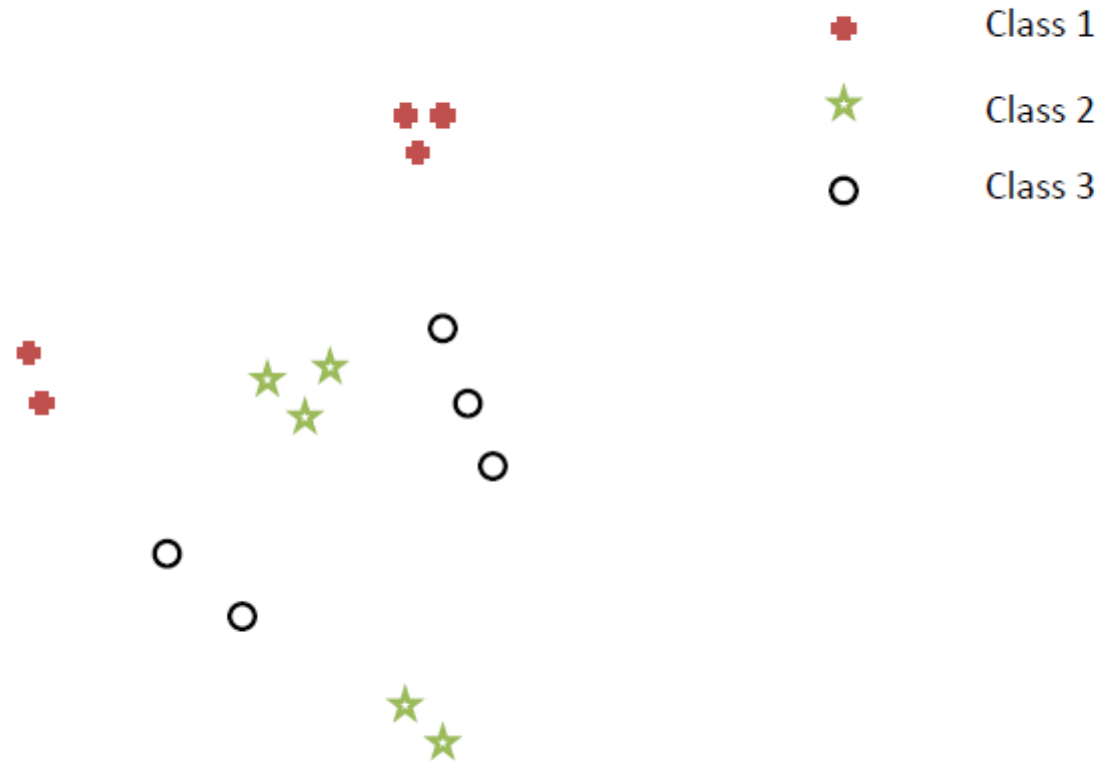
Relation with EXOR ...

x_1	x_2	o_1	o_2	$o = o_1 \vee o_2$
0	0	0	0	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

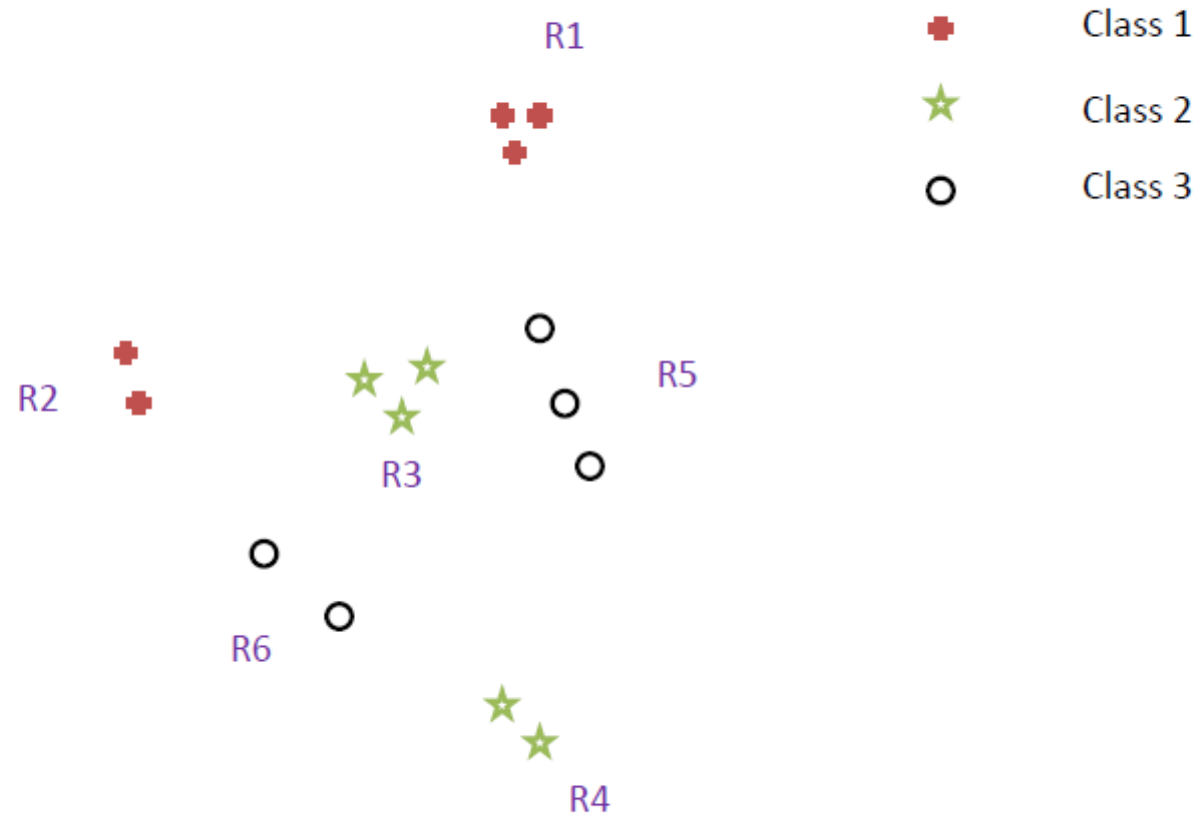
$$\Rightarrow o = o_1 \vee o_2 = x_1 \oplus x_2$$



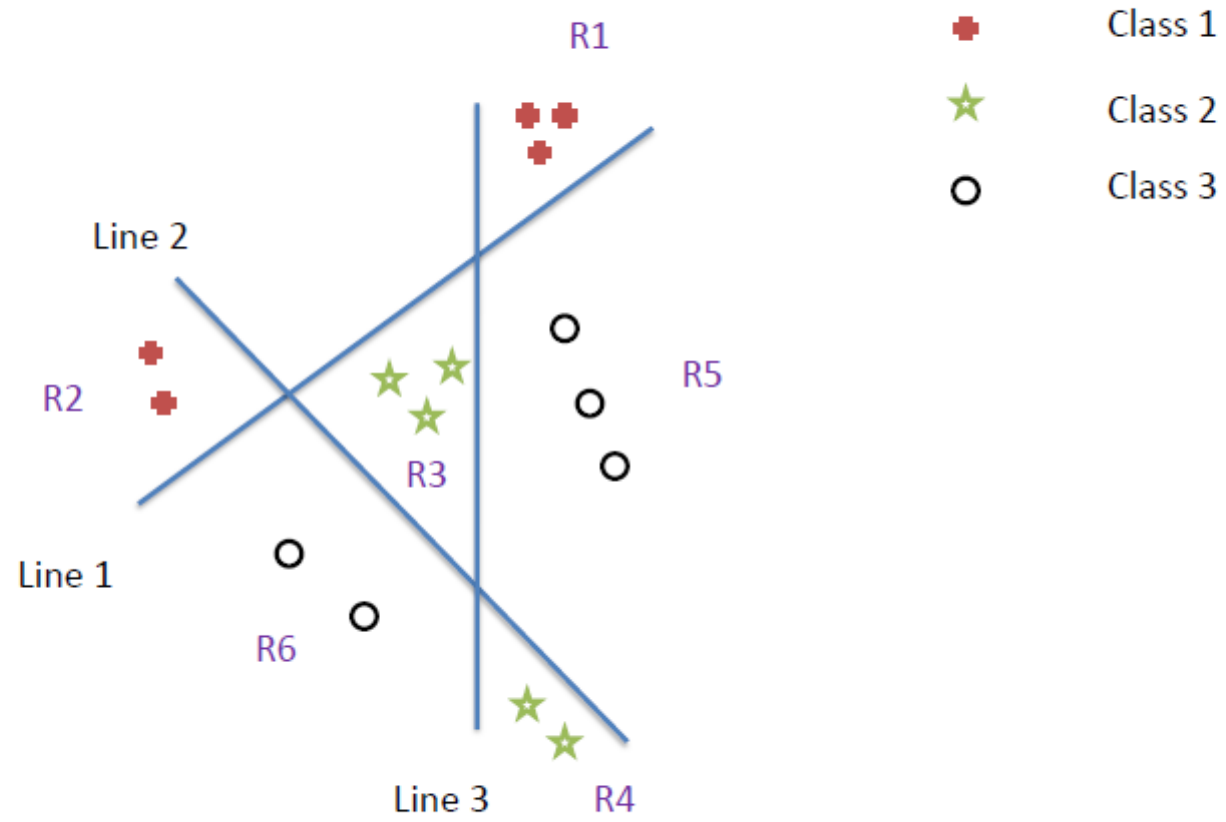
Sample Classification Problem :



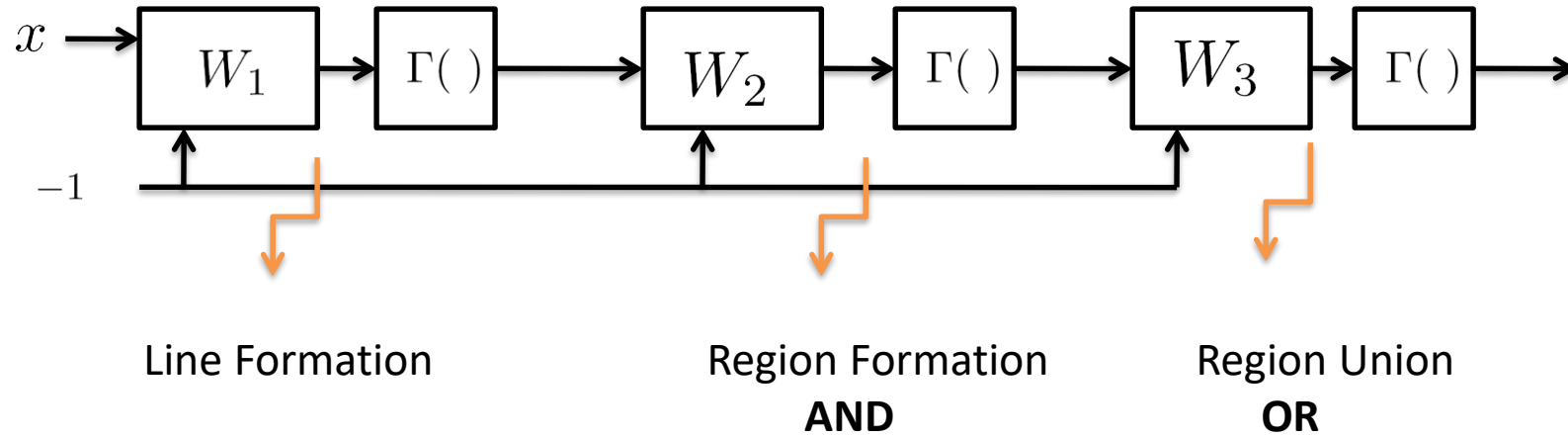
Linearly Separable Patterns :



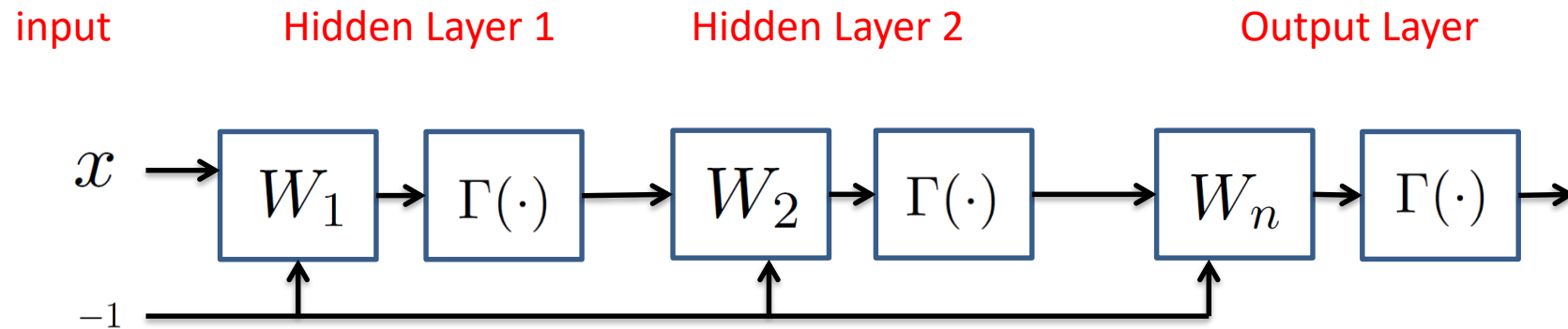
Linearly Separable Patterns :



At Most 3 layers are sufficient to classify Linearly Separable patterns...

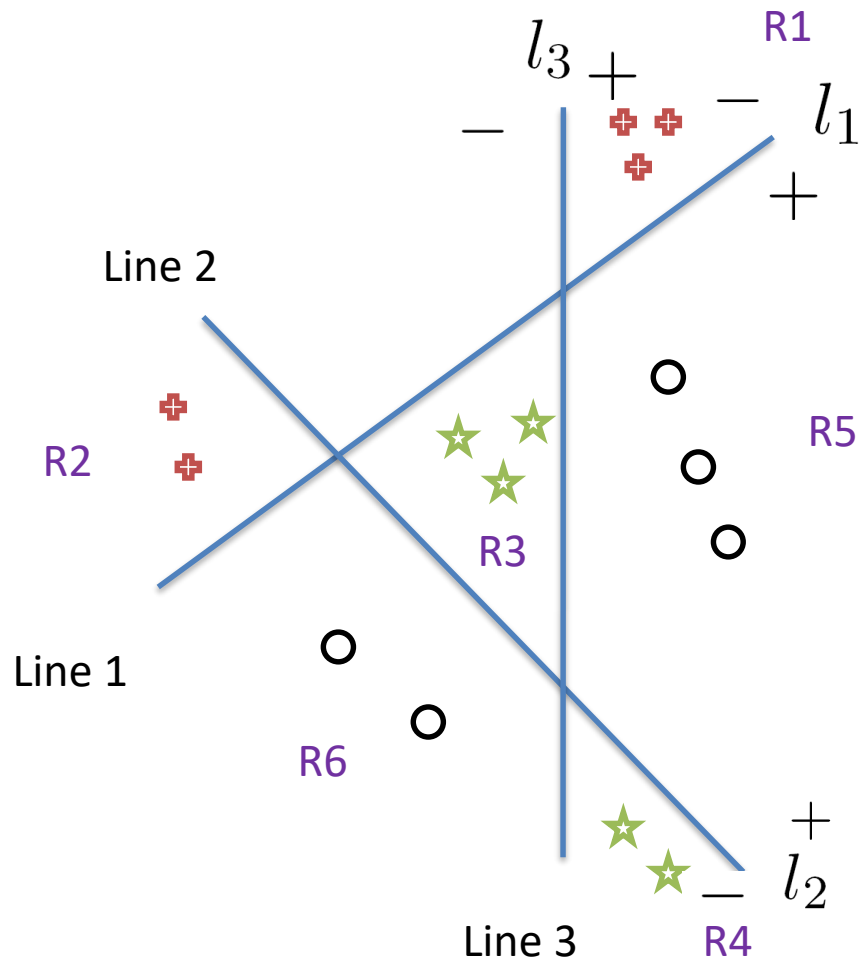


- First Layer **forms the lines** which separates the regions...
- Second Layer **forms the regions** by using **AND** operation...
- Third Layer **combines the regions** by using **OR** operation...



$$o = \Gamma(W_n(\Gamma(\dots W_2\Gamma(W_1x - \theta_1) - \theta_2)\dots) - \theta_n)$$

Could also use extended notation



$$l_1 : w_{11}x_1 + w_{12}x_2 - \theta_1 = 0$$

$$l_2 : w_{21}x_1 + w_{22}x_2 - \theta_2 = 0$$

$$l_3 : w_{31}x_1 + w_{32}x_2 - \theta_3 = 0$$

$$R1 : \bar{l}_1 \wedge l_3$$

$$R2 : \bar{l}_1 \wedge \bar{l}_2$$

$$R3 : l_1 \wedge l_2 \wedge \bar{l}_3$$

$$R4 : \bar{l}_2 \wedge l_3$$

$$R5 : l_1 \wedge l_2 \wedge l_3$$

$$R6 : l_1 \wedge \bar{l}_2 \wedge \bar{l}_3$$

$$\text{Class 1} : R1 \vee R2$$

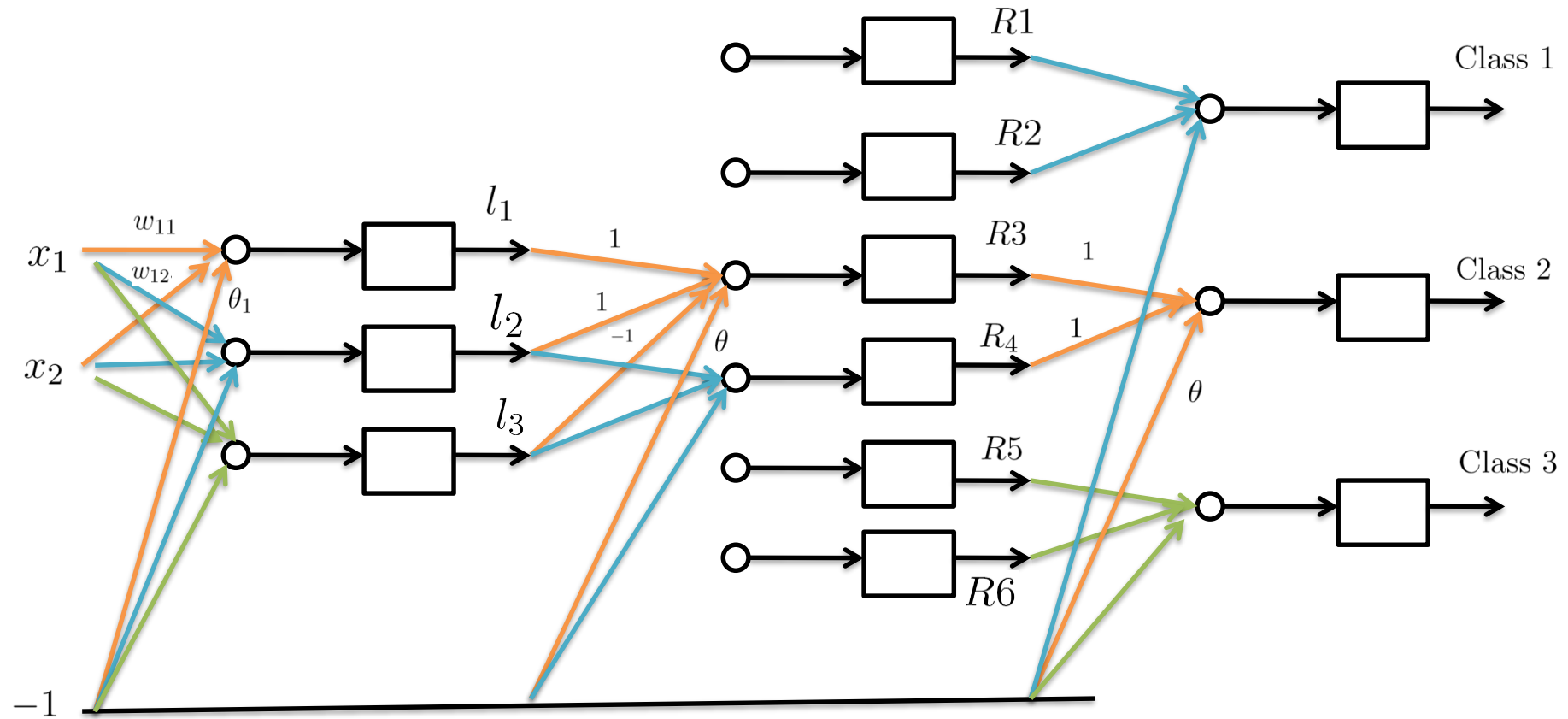
$$\text{Class 2} : R3 \vee R4$$

$$\text{Class 3} : R5 \vee R6$$

Line Weights

AND

OR



ANN Architectures

Mathematically, ANNs can be represented as weighted directed graphs. The most common ANN architectures are:

Single-Layer Feed-Forward NNs: One input layer and one output layer of processing units. No feedback connections (e.g. a Perceptron)

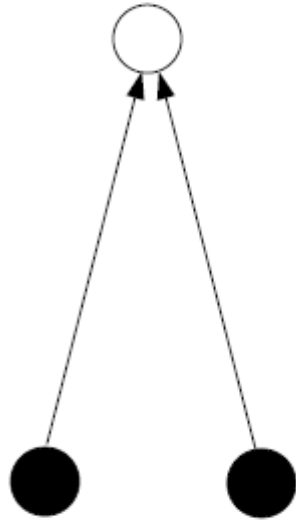
Multi-Layer Feed-Forward NNs: One input layer, one output layer, and one or more hidden layers of processing units. No feedback connections (e.g. a Multi-Layer Perceptron)

Recurrent NNs: Any network with at least one feedback connection. It may, or may not, have hidden units

Further interesting variations include: sparse connections, time-delayed connections, moving windows, ...

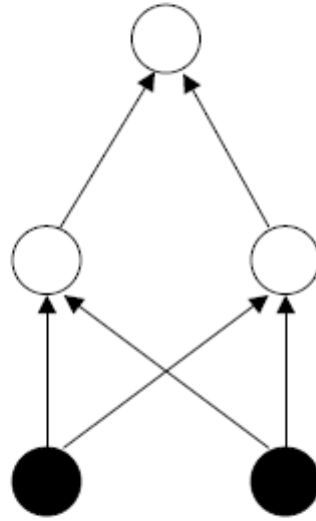
Examples of Network Architectures

Single Layer
Feed-forward



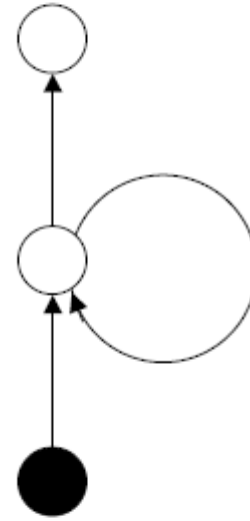
Single-Layer
Perceptron

Multi-Layer
Feed-forward



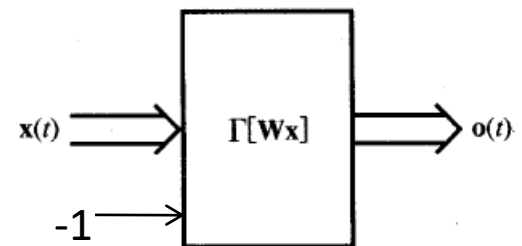
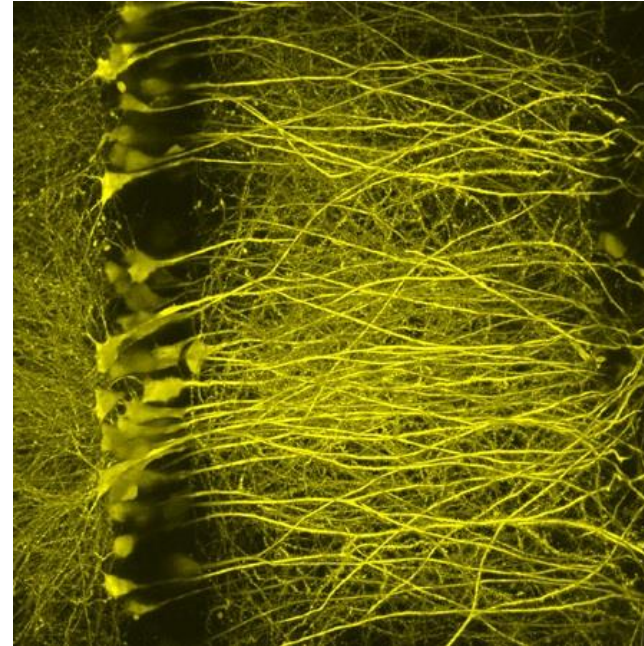
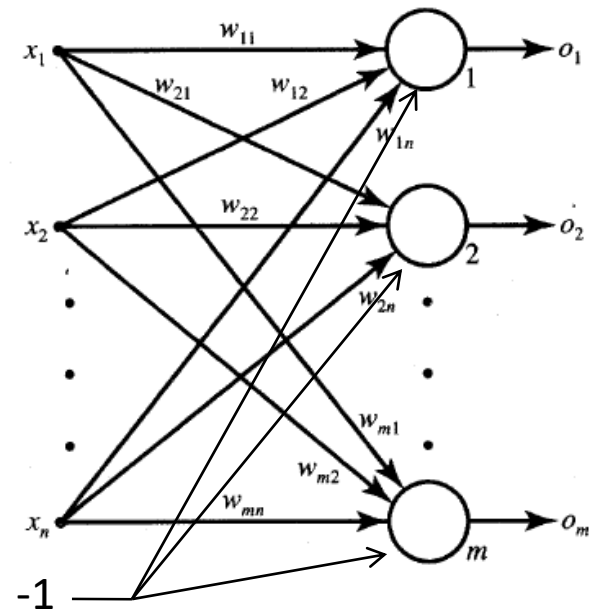
Multi-Layer
Perceptron

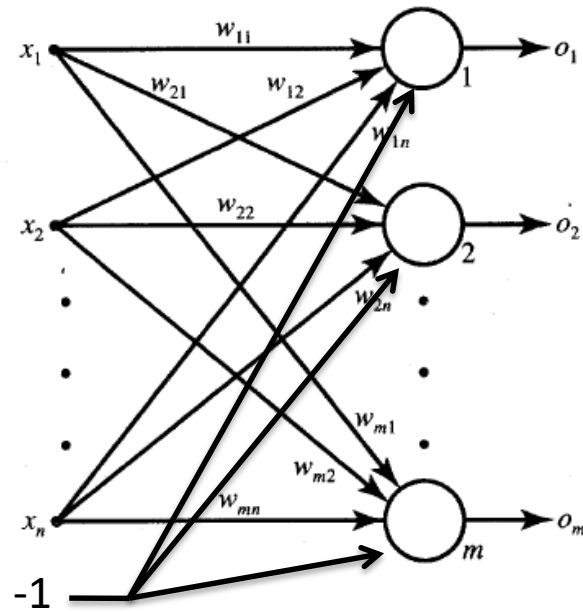
Recurrent
Network



Simple Recurrent
Network

- Single neuron is not sufficient !
- One Extension : **SINGLE LAYER NETWORK**





- mathematical equations?

componentwise equations

$$v_1 = w_{11}x_1 + w_{12}x_2 + \dots + w_{1n}x_n - \theta_1 \quad o_1 = f(v_1)$$

⋮

$$v_i = w_{i1}x_1 + w_{i2}x_2 + \dots + w_{in}x_n - \theta_i \quad o_i = f(v_i)$$

⋮

$$v_m = w_{m1}x_1 + w_{m2}x_2 + \dots + w_{mn}x_n - \theta_m \quad o_m = f(v_m)$$

Linear Sum

Output

vectorial notation

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad w_i = \begin{pmatrix} w_{i1} \\ w_{i2} \\ \vdots \\ w_{in} \end{pmatrix} \quad v = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_m \end{pmatrix} \quad o = \begin{pmatrix} o_1 \\ o_2 \\ \vdots \\ o_m \end{pmatrix}$$

input

i th neuron weight

linear sum

output

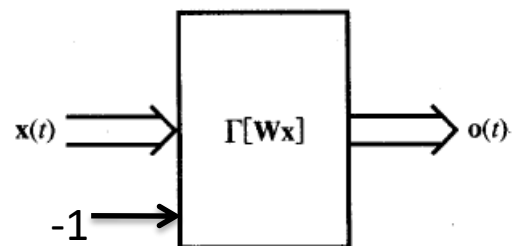
$$v_i = w_i^T x - \theta_i \quad o_i = f(v_i) \quad i = 1, 2, \dots, m$$



number of neurons

$$W = \begin{pmatrix} w_1^T \\ w_2^T \\ \vdots \\ w_m^T \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{pmatrix} \quad \theta = \begin{pmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_m \end{pmatrix}$$

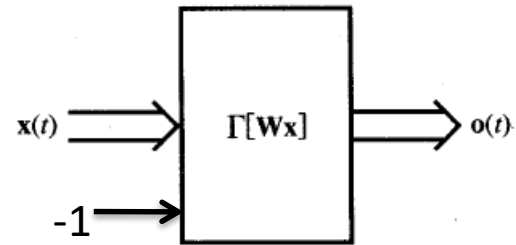
$$\Gamma(v) = \begin{pmatrix} f(v_1) & 0 & \dots & 0 \\ 0 & f(v_2) & \dots & 0 \\ & & \vdots & \\ 0 & 0 & \dots & f(v_m) \end{pmatrix}$$



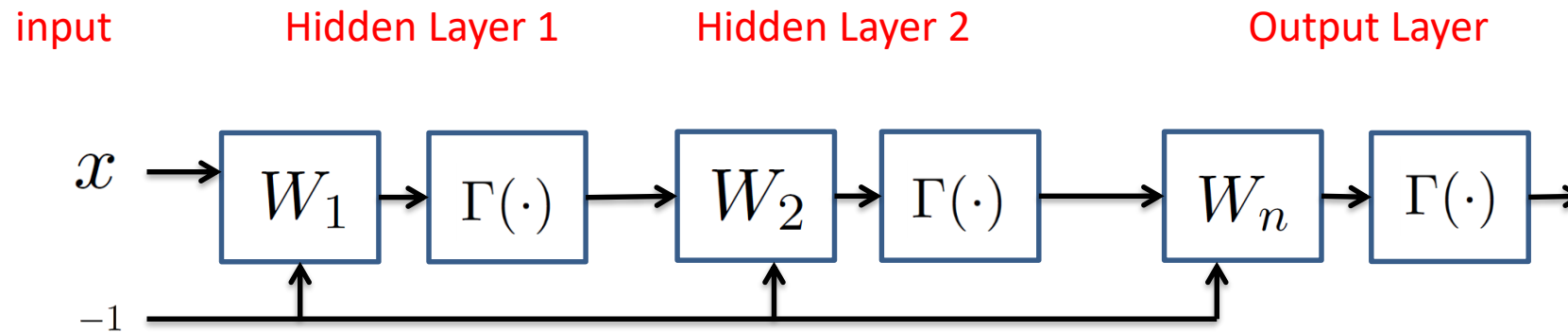
$$v = Wx - \theta \quad o = \Gamma(v) = \Gamma(Wx - \theta)$$

extended notation

$$W_e = [W \mid \theta] = \begin{pmatrix} w_{11} & w_{12} & \dots & w_{1n} & \theta_1 \\ w_{21} & w_{22} & \dots & w_{2n} & \theta_2 \\ w_{m1} & w_{m2} & \dots & w_{mn} & \theta_m \end{pmatrix} \quad x_e = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ -1 \end{pmatrix}$$



$$v = W_e x_e \quad o = \Gamma(v) = \Gamma(W_e x_e)$$



$$o = \Gamma(W_n(\Gamma(\dots W_2\Gamma(W_1x - \theta_1) - \theta_2)\dots) - \theta_n)$$

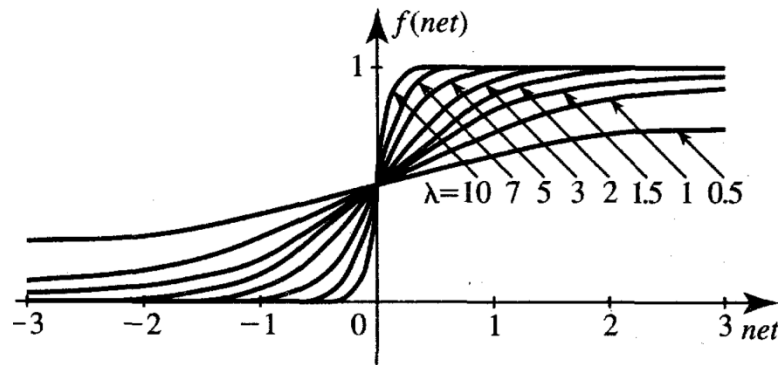
Could also use extended notation

Nonlinear Functions

- Unipolar Case $\rightarrow 0 \leq o \leq 1$

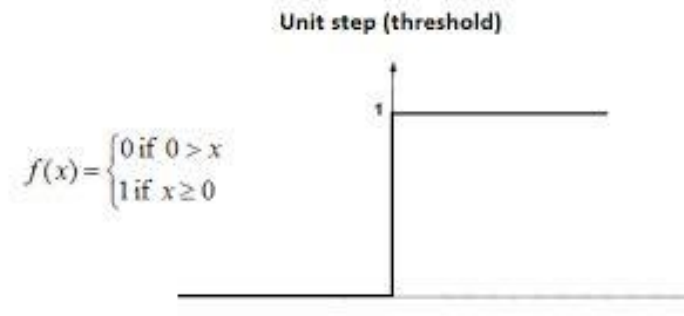
Unipolar Sigmoidal Function :

$$f(v) = \frac{1}{1+e^{-\lambda v}} \rightarrow \lambda > 0$$



Hard Limiter :

$$f(v) = \begin{cases} +1 & v \geq 0 \\ 0 & v < 0 \end{cases}$$



$$\lim_{\lambda \rightarrow \infty} (\text{unipolar sigmoidal}) = (\text{hard limiter})$$

Approximation !

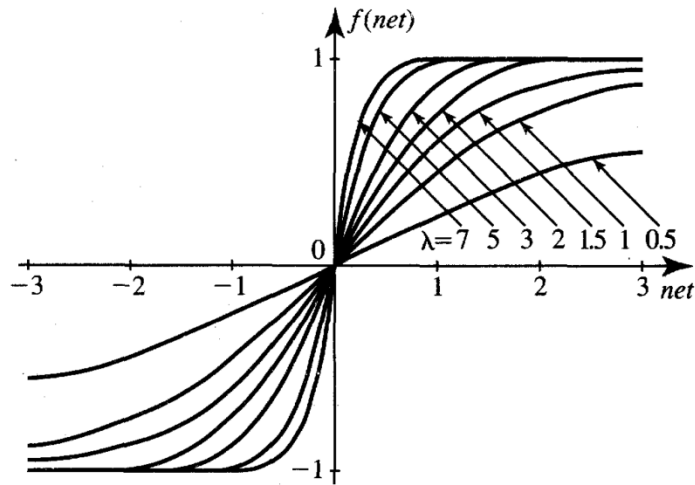
Bipolar Case $\rightarrow -1 \leq o \leq 1$

Bipolar Sigmoidal Function :

Hyperbolic tangent function

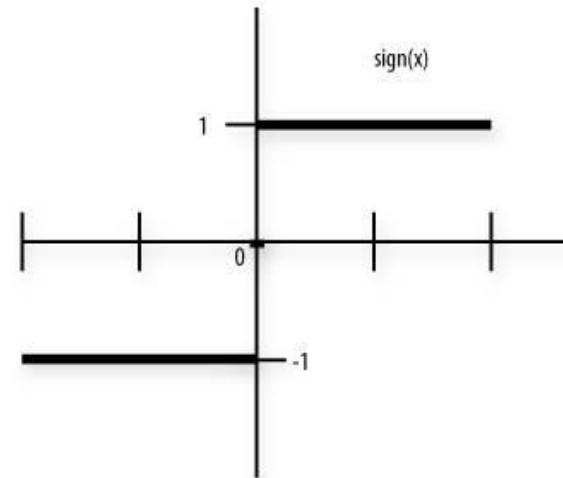
$$f(v) = \frac{1 - e^{-\lambda v}}{1 + e^{-\lambda v}} \rightarrow \lambda > 0$$

$$f(v) = \tanh(\lambda v)$$



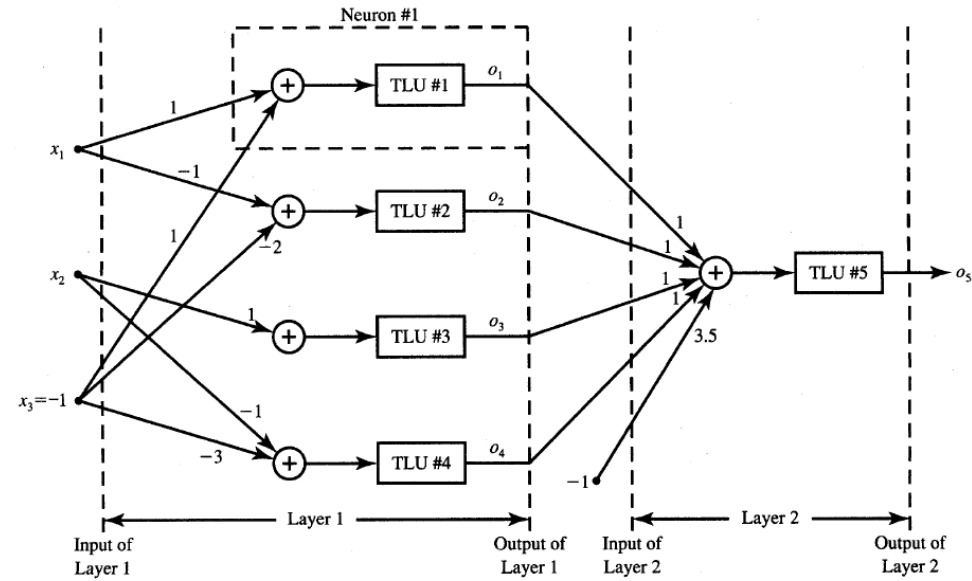
Signum Function

$$f(v) = \begin{cases} +1 & v \geq 0 \\ -1 & v < 0 \end{cases}$$

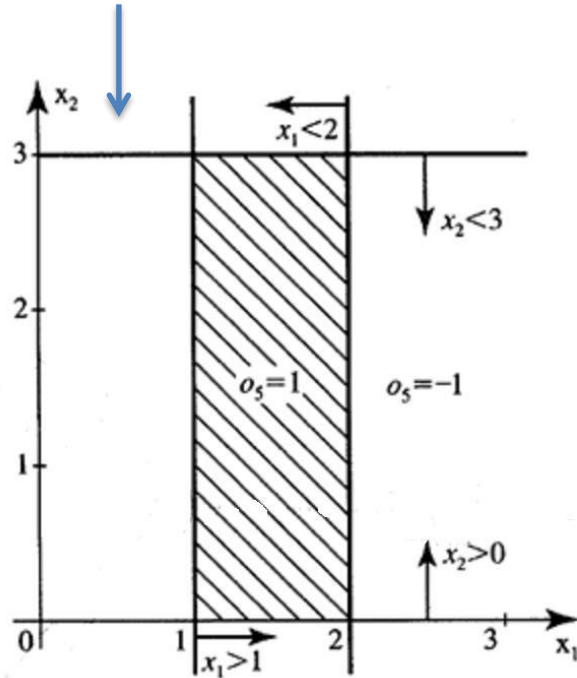


$$\lim_{\lambda \rightarrow \infty} (\text{bipolar sigmoidal}) = (\text{signum})$$

Approximation !



with signum



with bipolar sigmoidal

